

Parallelism & Concurrency Assignment 1 Report

By Neha Chakraborty (373384) & Guillaume Marie Lepin (381189)

Program A - Monte-Carlo estimation of Π (pi)

1) Program phases and what was parallelized

The 3 phases of this program were: **Initialization (serial)**: parse arguments, start the timer, set up OpenMP region. **Sampling loop (parallel)**: generate random samples and count points in the circle. **Reduction + final computation**: reduce thread-local counts into a single counter and compute the final estimate. We parallelized the sampling loop, because each sample is independent. The only shared value is the counter (points_inside), which we combined using an OpenMP reduction to avoid race conditions.

2) Dominant operations per phase

Initialization phase: negligible; dominated by constant-time setup (argument parsing and timer). **Sampling loop**: dominated by random number generation (two calls per sample) and the arithmetic check of $r^2 = x^2 + y^2$. We considered random generation as the dominant cost, because it happens for every sample and is costlier than a few floating-point operations. **Reduction/final computation**: dominated by the reduction of per-thread partial results, but this cost is small compared to the main loop for large sample counts.

3) Performance critical arguments and asymptotic time

Let N = number of samples and T = number of threads

Initialization: does not depend on N . Time: $O(1)$ **Sampling loop**: runs N iterations and each iteration does constant work (random number generation + arithmetic). Serial time: $O(N)$ Parallel time: $O(N/T)$ **Reduction + final computation**: reduction over threads. Time: $O(T)$

Overall: Serial time: $O(N)$ Parallel time: $O(N/T)$

4) Predicted speedup and scaling

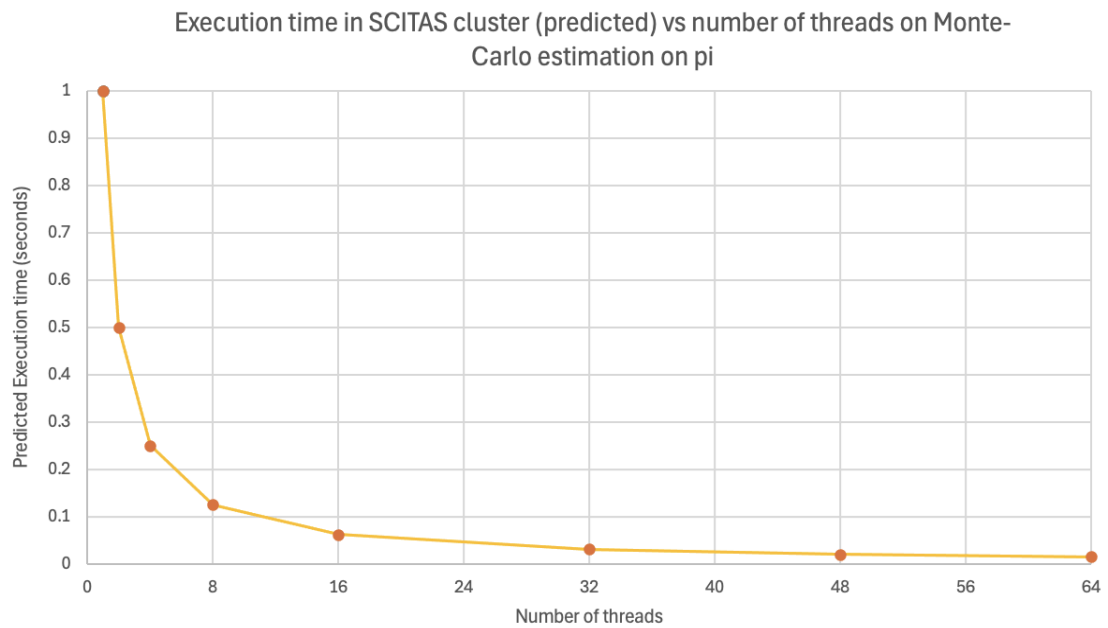
Let $S(T)$ = execution time with 1 thread / execution time with T threads and

$\text{Time}(T)$ = execution time using T threads

If we ignore hardware limits, the sampling work divides perfectly among threads. That gives a speedup close to: **$S(T) \approx T$ and $\text{Time}(T) \approx \text{Time}(1) / T$** .

Estimated speedups: 1 thread $\Rightarrow 1x$, 2 threads $\Rightarrow 2x$, 4 threads $\Rightarrow 4x$, 8 threads $\Rightarrow 8x$, 16 threads $\Rightarrow 16x$, 32 threads $\Rightarrow 32x$, 48 threads $\Rightarrow 48x$, 64 threads $\Rightarrow 64x$.

Estimated execution time vs number of threads: function $\Rightarrow 1/(\# \text{ of threads})$

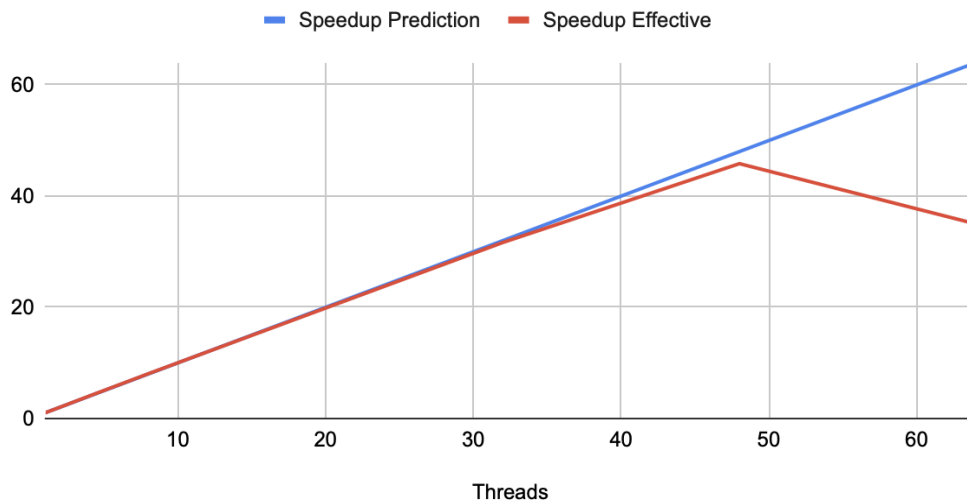


5) Measured results

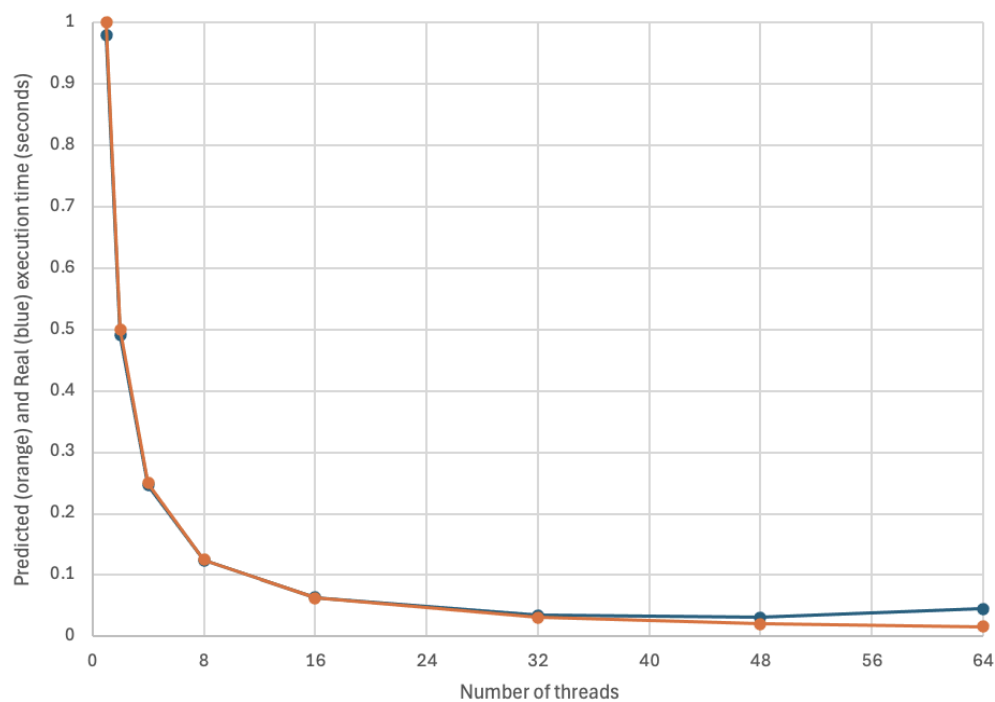
We fixed the number of samples to 10^9 to ensure a consistent workload across experiments. The program was compiled on the cluster to ensure compatibility with the execution environment. We used a single node and a single task per node with up to 64 CPUs per task.

Number of threads	Execution time (seconds)	Speedup of multi-threaded program
1	9.8020	1.00000
2	4.9040	1.99840
4	2.4540	3.97570
8	1.2290	7.91600
16	0.6152	15.4234
32	0.3109	38.4635
48	0.2182	31.4002
64	0.2593	21.7102

Speedup Prediction and Speedup Effective VS. Number of Threads



Execution time in SCITAS cluster (predicted vs real) vs number of threads on Monte-Carlo estimation of pi



Predicted execution time (orange) vs Real execution time (blue)

6) Measured vs predicted speedups

The measured speedup closely follows the ideal linear prediction up to around 32 threads, but deviates at higher thread counts. Beyond this point, the speedup begins to decrease, with performance worsening at 64 threads. This is mainly due to hardware limitations, such as a limited number of physical cores, leading to contention between threads. Additionally, increased pressure on memory bandwidth and cache, as well as thread management overhead, prevent the program from achieving ideal scaling.

Program B - Monte-Carlo integration of integral

1) Program phases and what was parallelized

As with Program A, Program B can be divided into three main phases.

The first phase is **initialization** and remains **serial**. It consists of checking the validity of the input parameters, initializing the variables, and setting up the OpenMP execution environment. This phase prepares the program for the numerical computation but does not represent a significant part of the total execution time.

The second phase is the **sampling loop**, which is the **parallelized** part of the program. In this phase, each thread generates random values in the integration interval and evaluates the function at those sampled points. These evaluations are then used to estimate the value of the integral through the Monte Carlo method. This loop is the computational core of the program and therefore the main target for parallelization.

The third phase is the **reduction and return** phase. After each thread has computed its own partial contribution to the integral, OpenMP combines these local results into a final global value using a reduction operation. The program then returns the computed integral. This final phase is effectively serial from the programmer's perspective, although the reduction itself may internally involve multiple steps.

The sampling loop is parallelized because its iterations are independent: each sample can be generated and evaluated without interacting with the others. The only shared quantity is the global integral estimate, which is safely handled using an OpenMP reduction clause to avoid race conditions.

2) Dominant operations per phase

The **initialization phase** runs in constant time, since it only involves simple tasks such as checking arguments, computing basic constants, and configuring OpenMP. Its computational cost is negligible compared with the rest of the program.

The **sampling phase** dominates the execution time. Each iteration performs random number generation, maps the generated value into the integration interval, evaluates the function, and contributes to the accumulated result. Among these operations, **random number generation** and **function evaluation** are the most expensive. Since this work is repeated for every sample, this phase represents the main computational cost of the program.

The **reduction and return phase** has a relatively small cost. Its runtime depends mainly on the combination of the partial sums produced by each thread. In practice, this overhead is much smaller than the cost of the sampling loop, especially when the number of samples is very large.

3) Performance critical arguments and asymptotic time

Same as for Program A, we set N = number of samples and T = number of threads.

Initialization runs in constant time so it is $O(1)$. **Loop Section** runs N times and each iteration runs in a constant time. However, it is separated into T threads. The final runtime is $O(N/T)$ if the process is parallelized and $O(N)$ if it is serialized. **The Reduction and Return** phase runs depending on the number of threads because of their reduction. This part is $O(T)$.

Overall: Serial Time: $O(N)$. Parallel Time: $O(N/T)$.

4) Predicted speedup and scaling

As we did for program A:

We set $S(T)$ = execution time with 1 thread / execution time with T threads and

$\text{Time}(T)$ = execution time using T threads.

We also assume that Initialization and Reduction both take an arbitrary 0.1 U.A if the whole program runs in 1 U.A for 1 thread.

As previously stated: If we ignore hardware limits, the sampling work divides perfectly among threads. That gives a speedup close to: $S(T) \approx T$ and $\text{Time}(T) \approx \text{Time}(1) / T$.

Estimations:

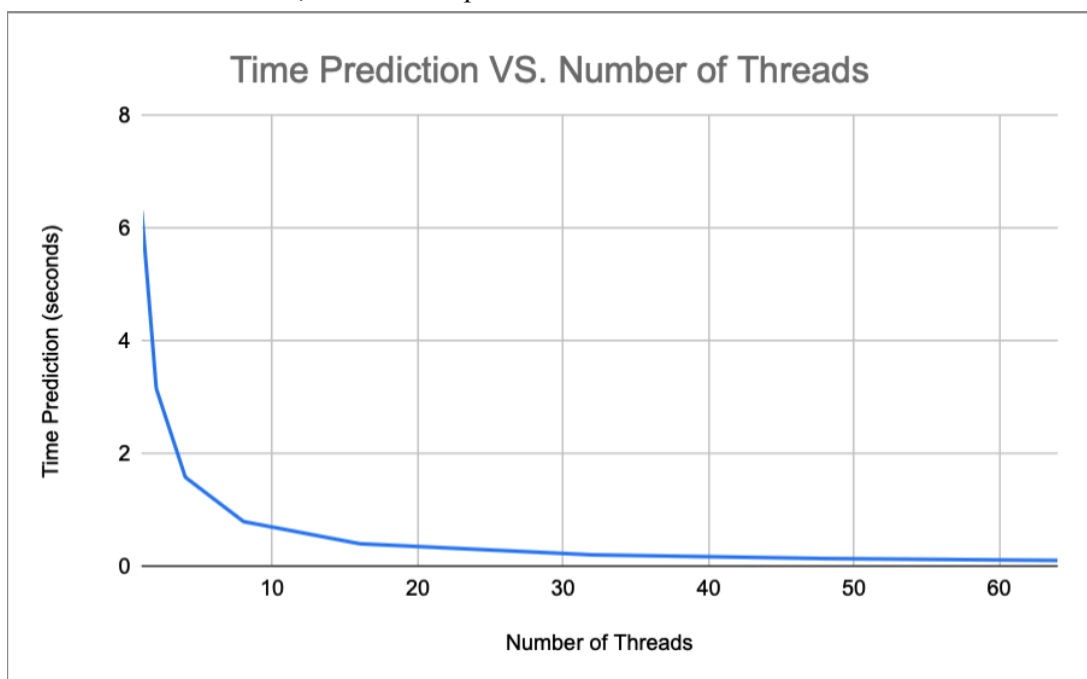
X-Axis = Number of threads

Y-Axis = Execution time compared to original value.

We consider the runtime for 1 thread as the real runtime (4.995 seconds)

The function that determines the graph takes the following input: `./integral 1 1000000000 5 9`.

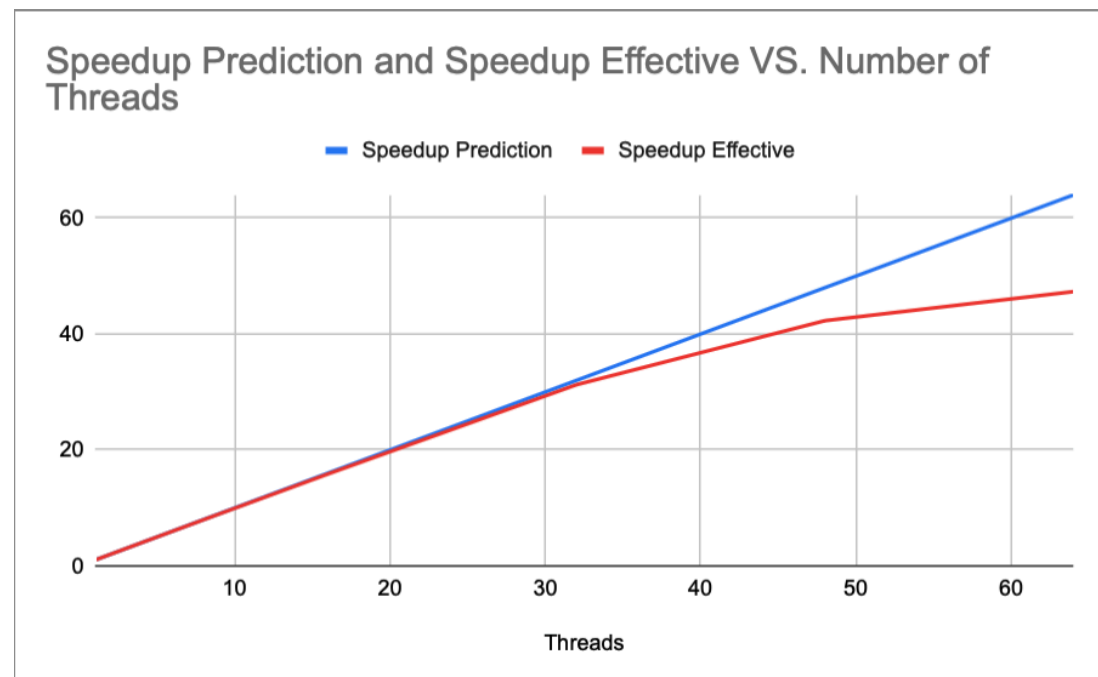
From the result of 4.995s, we drew the prediction:

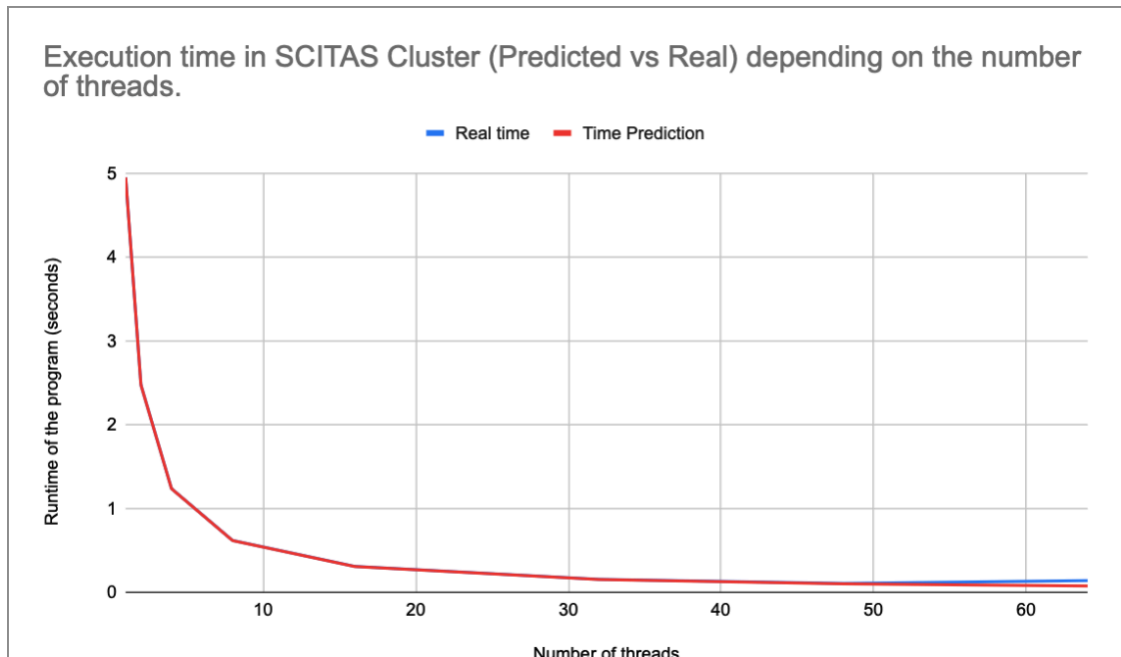


5) Measured results

To establish these results, we run `./integral thread_n 1000000000 5 9`.

Number of threads	Execution time (seconds)	Speedup of multi-threaded program
1	6.3040	1.0000
2	3.1560	1.9970
4	1.5790	3.9920
8	0.7908	7.9700
16	0.3976	15.855
32	0.2017	31.254
48	0.1490	42.308
64	0.1332	47.320





6) Measured vs predicted speedup

The predicted speedup deviates from the measured performance due to parallel overheads that become significant at higher thread counts. These overheads include thread management, scheduling, synchronization (reduction), and increased pressure on shared resources such as memory bandwidth.

As the number of threads grows, the amount of useful work per thread decreases, while overhead remains relatively constant or increases. This leads to diminishing returns and eventually performance degradation.

This effect is clearly visible near 64 threads, where the runtime increases instead of decreasing, indicating that the application has surpassed its scalability limit.

Program C - True/False Sharing Optimization

1) True/False Sharing

After the first improvement of the program, by naive parallelization of the loops, there is true false sharing. This is explained by the sharing of an array called histogram that gets invalidated for the other caches after each modification.

```
int perform_buckets_computation(int num_threads, int num_samples, int
num_buckets) {
    volatile int *histogram = (int*) calloc(num_buckets, sizeof(int));

    #pragma omp parallel num_threads(num_threads){
        unsigned int thread_id = omp_get_thread_id();
        rand_gen generator = init_rand(thread_id);

        #pragma omp for reduction(+:histogram[:num_buckets])
        for(int i = 0; i < num_samples; i++){
            int val = next_rand(generator) * num_buckets;
            histogram[val]++;
        }
        free_rand(generator);
    }
    return 0;
}
```

2) Improvement for True/False Sharing

In order to improve performances, we decided to create a local histogram for each thread that allows them to modify it without interfering with other threads modifications. It reduces cache misses.

At the end, we put everything together using parallelized for loops and atomic operations.

3) Performances

Here are the performances of the algorithm before and after true/false sharing improvement, the performance of the program after the reduction of true/false sharing is significantly better.

10^8 operations executed each time:

Simple parallelization		Fully improved	
Number of threads	Execution time (s)	Number of threads	Execution time (s)
1	0.84850	1	0.53310
2	1.8140	2	0.26720
4	1.7390	4	0.13380

8	2.1490	8	0.06753
---	--------	---	---------

4) Histogram Size Effect

The bigger the histogram, the less chance True/False sharing happens. If the histogram is too small, the same line will be modified more often. However, there is a certain threshold for performance. If the histogram is too large, statistically, the lines in the cache will be changed every time and it won't be so optimal.

Program D - Parallel optimization of Reducing Matrix Multiplication (RMM)

1) Parallelization and optimization of the RMM algorithm

Optimization 1: Parallelization of the outer loop (idx)

We parallelized the outer loops using `#pragma omp parallel for collapse(2)`. This distributes the 2D iteration space over blocks (i, j) across threads. Each thread processes independent tiles of the output matrix C, ensuring no write conflicts since each (i1, j1) corresponds to a unique element of C. This approach increases parallelism compared to a single loop and improves load balancing across threads.

Optimization 2: Cache pointers to the two A rows used by a given output row

We store `int *A0 = matA[2 * i1]; int *A1 = matA[2 * i1 + 1];`. This reflects that each row of C depends on two consecutive rows of A. These pointers are reused inside the inner k loop, avoiding repeated indexing into matA. This reduces address computation overhead and ensures efficient sequential access to A.

Optimization 3: Cache B column indices

We compute `col0 = 2 * j1` and `col1 = col0 + 1` outside the inner loop. Each column of C corresponds to two consecutive columns of B. By reusing these indices inside the k loop, we avoid recomputing them and reduce arithmetic overhead.

Optimization 4: Loop tiling (blocking) and loop unrolling of the computation

We introduced loop tiling using a block size (`BLOCK_SIZE = 16`) over the (i, j) dimensions. This divides the computation into smaller submatrices, improving cache locality when accessing matC and reusing portions of A and B within each tile.

Inside the innermost loop, we manually unroll the computation by loading values once:

```
int a0_val = A0[k];
int a1_val = A1[k];
int b_col0 = matB[k][col0];
int b_col1 = matB[k][col1];
```

and computing:

```
sum += a0_val * b_col0;
sum += a0_val * b_col1;
sum += a1_val * b_col0;
sum += a1_val * b_col1;
```

This removes the small inner loops over `aoff` and `boff`, eliminates conditional checks, and reduces repeated memory accesses. It keeps the innermost loop simple and focused on arithmetic operations, improving performance.

2) Ways of splitting work among threads and which give more locality

There are three work partitions for the (idx, jdx) space:

- 1) Split by idx only (parallelize outer loop)
- 2) Split by (idx, jdx) with collapse(2) (what we used)
- 3) Split by jdx (parallelize columns)

We parallelize over both i and j using collapse (2), which distributes 2D tiles of the computation across threads. Each thread processes independent blocks of matrix C which increases the number of parallel tasks and improves load balancing, especially when matrix dimensions are large.

By introducing blocking, each thread works on a small tile of C and repeatedly accesses the corresponding rows of A (A0, A1) and columns of B(col0, col1) within that tile. This helps maintain locality despite using collapse(2), as data is reused within each block before moving to another region.

Parallelizing only over i (rows) would improve locality for matrix A, since each thread would reuse the same rows (A0, A1) across all j. However, it provides fewer parallel iterations (M/2), which may limit scalability compared to collapse(2). Parallelizing over j (columns) was also considered, but this would result in poorer locality for matrix A, since multiple threads would access the same rows simultaneously without reusing the data.

3) Load balancing issues

The workload per output element is uniform, so there are no inherent load balancing issues. Each (i1, j1) computation performs the same amount of work, and the iterations are distributed across threads using #pragma omp parallel for collapse(2). This creates a larger number of independent tasks compared to parallelizing a single loop, improving load balancing.

With static scheduling, the 2D iteration space over (i, j) blocks is evenly divided among threads. Since the matrix dimensions are large and the blocks are of fixed size (BLOCK_SIZE), each thread receives a similar amount of work. This ensures that all threads remain well utilized throughout execution.

4) Experiments to isolate effects of each optimization / thread organization

To isolate the effect of each optimization, we implemented the optimizations cumulatively:

Version	Description
Baseline	Original implementation
Optimization 1	Baseline + outer loop parallelization
Optimization 2	Optimization 1 + cached A row pointers
Optimization 3	Optimization 2 + cached B column indices and values
Optimization 4	Optimization 3 + loop tiling & unrolling

Each configuration was executed several times and the median execution time was recorded. We used a single node with one task per node and multiple CPUs per task, as the program

relies on OpenMP shared-memory parallelism. The number of threads was varied in the set {1, 2, 4, 8, 16, 32}, while keeping the matrix size fixed at $1024 \times 1024 \times 1024$ to ensure a fair comparison across experiments.

The execution times (in seconds) for each optimization stage are:

Threads	Baseline	Opt 1	Opt 2	Opt 3	Opt 4
1	0.90280	0.90190	0.53370	0.53650	0.48640
2	0.89980	0.44980	0.26670	0.26730	0.24360
4	0.89910	0.22830	0.13940	0.13980	0.12310
8	0.90060	0.11610	0.07002	0.06966	0.06176
16	0.90400	0.05917	0.03555	0.03504	0.03182
32	0.90280	0.03075	0.01841	0.01857	0.01666

Optimization 1 shows a significant improvement when increasing the number of threads. Compared to the baseline, the execution time decreases almost proportionally to the number of threads (e.g., from ~ 0.90 s to ~ 0.03 s at 32 threads). This demonstrates that the algorithm is highly parallelizable and that distributing work across threads is the primary source of performance improvement. The good scalability is due to uniform workload distribution and sufficient parallelism, further enhanced by using `collapse(2)` which increases the number of parallel tasks.

Optimization 2 introduces a major improvement even at a single thread (0.90 s $\rightarrow$ 0.53 s), indicating that this optimization primarily reduces computational overhead rather than improving parallelism. By caching pointers to rows of A, it reduces indexing costs and improves memory access efficiency. Optimization 3 shows a small improvement over Optimization 2. By caching column indices of B, it eliminates redundant arithmetic operations and slightly improves the efficiency of the inner loop.

Optimization 4 provides the largest overall improvement when combined with previous optimizations. The introduction of loop tiling (`BLOCK_SIZE = 16`) significantly improves cache locality by ensuring that data from A and B is reused within small blocks before moving to new regions. Additionally, manual unrolling of the inner computation reduces loop overhead and minimizes repeated memory accesses. Together, these optimizations reduce execution time by approximately 50% compared to the baseline in the single-thread case (0.90 s $\rightarrow$ ~ 0.49 s), and further improve performance at higher thread counts.

5) Execution time graph for threads {1,2,4,8,16,32} with $M=N=K=1024$ on SCITAS cluster

